

Tut 1 CS3210

Overview

- `generateLargeArray` : fills a `vector<int>` of length `size` with `rand() % max_value` , so each element is in `[0, max_value)` .
- `my_sort` : **bubble sort**. The inner loop compares adjacent pairs and swaps if out of order; the bound `n - i - 1` shrinks each pass because after pass `i` , the `i` largest elements have "bubbled" to their final positions at the tail. Ascending order.
- `main` : generates a 10,000-element array, sorts it, prints the last 5 (i.e. the 5 largest).

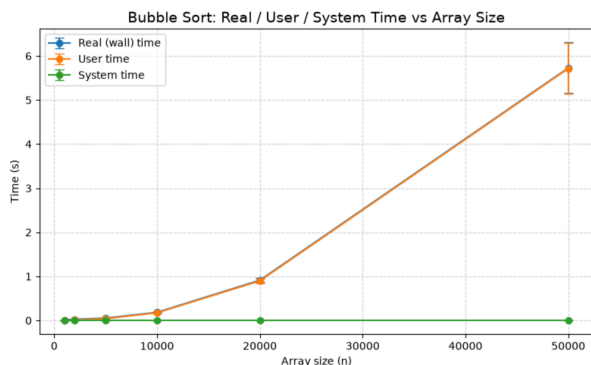
Description

```
int main(int argc, char** argv) {  
    // These parameters can be changed  
    size_t ARRAY_SIZE = 10000;  
    size_t MAX_VALUE = 100000;  
  
    if (argc > 1) ARRAY_SIZE = std::strtoull(argv[1], nullptr, 10);  
    if (argc > 2) MAX_VALUE = std::strtoull(argv[2], nullptr, 10);  
}
```

Updated asdf.cpp to take in variable array_size and max_value

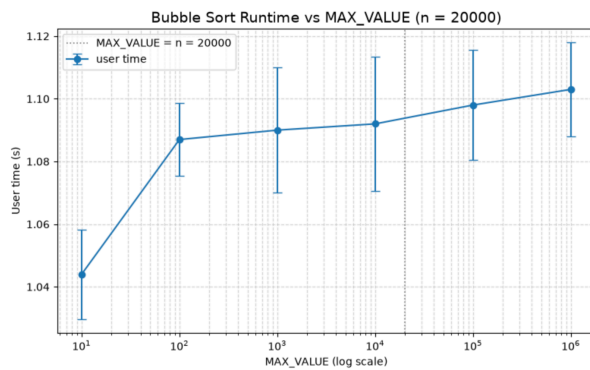
Array size scaling

Wrote a bash script to measure how the change in the Array size affected the user time, real time and the system time. The system time is mostly constant across the runs since it is single threaded with no context switch overhead. The user time grows quadratically since the time complexity of the Bubble sort is $O(n^2)$.



Max value scaling

Increasing the max possible random value did not significantly increase the user time. Other than the jump between `max_value = 10` to `max_value = 100`. This is because the number of swaps between values is small when the max value is small.



Optimisation

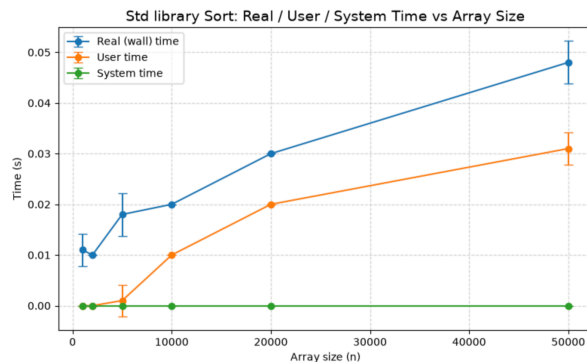
```
std::sort(largeArray.begin(), largeArray.end());
```

Utilised standard library sort instead of the bubble sort. Theoretical time complexity of `std::sort` is $O(n \log n)$.

Unlike bubble sort, where $\text{real} \approx \text{user}$ (sort dominated runtime), `std::sort` shows $\text{real} > \text{user}$ by ~50% at $n=50000$.

The sort is now so fast that whole-process time is dominated by array generation and process overhead rather than the sort itself.

`/usr/bin/time` — appropriate for the multi-second bubble sort — lacks the resolution and isolation to measure `std::sort` cleanly at these sizes.



Appendix

Script to measure Run time vs variable array size - https://github.com/limzishen/Everything-Bank/blob/master/Classes%20and%20stuff/CS3210/T1/run_time.sh

Script to measure Run time vs variable max value - https://github.com/limzishen/Everything-Bank/blob/master/Classes%20and%20stuff/CS3210/T1/max_value_run_time.sh

Notebook to generate data visualisation - https://github.com/limzishen/Everything-Bank/blob/master/Classes%20and%20stuff/CS3210/T1/Data_visualisation.ipynb

Results for run time vs variable array size - <https://github.com/limzishen/Everything-Bank/blob/master/Classes%20and%20stuff/CS3210/T1/results.csv>

Results for run time vs variable max value - https://github.com/limzishen/Everything-Bank/blob/master/Classes%20and%20stuff/CS3210/T1/results_maxval.csv

Results for run time vs variable array size after sort optimisation - https://github.com/limzishen/Everything-Bank/blob/master/Classes%20and%20stuff/CS3210/T1/optimised_sort_results.csv